Unit 8: JavaFX 4 hrs.

8.1 Introduction to JavaFX
8.2 JavaFX Architecture
8.3 JavaFX Application Basics-Line, rectangle and text

Introduction to JavaFX

JavaFX is a software platform for creating and delivering desktop applications, as well as rich web applications that can run across a wide variety of devices.

JavaFX has support for desktop computers and web browsers on **Microsoft Windows, Linux, and macOS**, as well as mobile devices running **iOS and Android**.

Architecture of JavaFX

JavaFX is a software platform for creating and delivering desktop and web applications with a rich user interface. It provides a lightweight, hardware-accelerated UI framework that runs across multiple devices, including **Windows, Linux, macOS, iOS, and Android**.

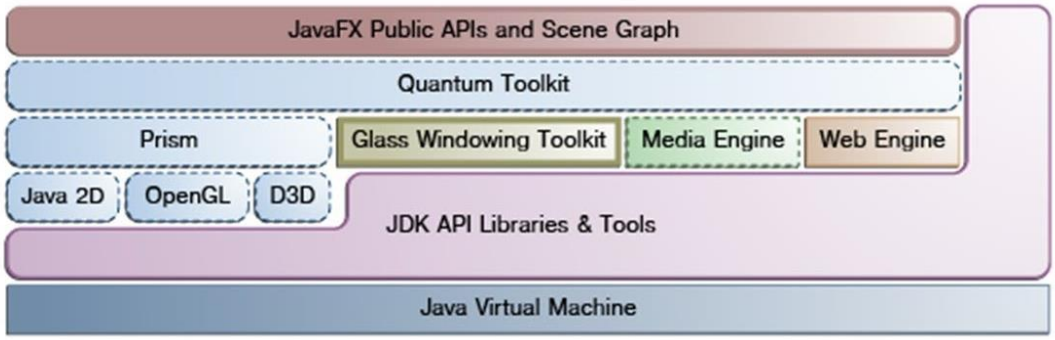


Figure 1: Architecture of JavaFX

JavaFX Public APIs and Scene Graph

- The scene graph is a hierarchical structure of UI elements.
- Developers use Java APIs to build and manipulate UI components.

Quantum Toolkit

- Bridges JavaFX APIs with lower-level components.
- Ensures smooth rendering and event handling.

Prism (Rendering Engine)

- High-performance graphics engine that supports Direct3D, OpenGL, or Java2D pipelines.
- Responsible for rendering UI elements efficiently.

Glass Windowing Toolkit

- Manages native OS windowing operations.
- Handles user input events (keyboard, mouse, touch).

Media Engine

- Provides audio and video playback functionality.
- Supports various media formats.

Web Engine

- Allows embedding of web content inside JavaFX applications.
- Renders HTML, CSS, and JavaScript.

JavaFX Application Basics-Line, rectangle and text

In **JavaFX**, we can draw **lines, rectangles, and text** using the javafx.scene.shape and javafx.scene.text packages.

For example.

```
import javafx.application.Application;
import javafx.stage.Stage;
public class App extends Application{
        @Override
        public void start(Stage stage){
                stage.setTitle("My First App");
                stage.show();
        }
        public static void main(String args[]){
                launch(args);
        }
}
```

Output:

Drawing a Line

Line is a part of JavaFX. The Line class represents a line in a 2D space. Use the Line class to draw a line by specifying the start and end coordinates.

Constructor

Line(): Creates a new instance for line
Line(double startX, double startY, double endX, double endY): Creates a new Line with specified starting and ending point

Methods

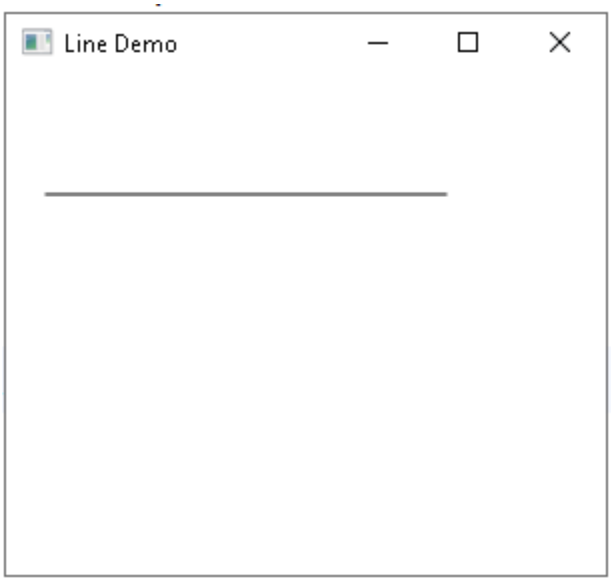
getEndX() returns the x coordinate for the end point
getEndY() returns the y coordinate for the end point
getStartX() returns the x coordinate for the start point
getStartY() returns the y coordinate for the start point
setEndX(double value) sets the x coordinate for the end point
setEndY(double value) sets the y coordinate for the end point
setStartX(double value) sets the x coordinate for the start point
setStartY(double value) sets the y coordinate for the start point

Example,

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.shape.Line;
import javafx.stage.Stage;

public class DrawLine extends Application {
    @Override
    public void start(Stage stage) {
        Pane pane = new Pane();
```

```
        Line line = new Line(20, 60, 220, 60);
        line.setStrokeWidth(1);

        pane.getChildren().add(line);
        Scene scene = new Scene(pane, 300, 250);
        stage.setTitle("Line Demo");
        stage.setScene(scene);
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:



Drawing a Rectangle

Use the Rectangle class to define a rectangular shape. Rectangle class is a part of JavaFX. Rectangle class creates a rectangle with specified width and height and position.

By default Rectangle has sharp corners but the edges can be rounded by applying a arc height and width.

Constructor:

Rectangle(): creates a empty instance of rectangle
Rectangle(double w, double h): creates a rectangle with a specified width and height
Rectangle(double x, double y, double w, double h): creates a rectangle with a specified width and height and position
Rectangle(double w, double h, Paint f): creates a rectangle with a specified width and height and fill

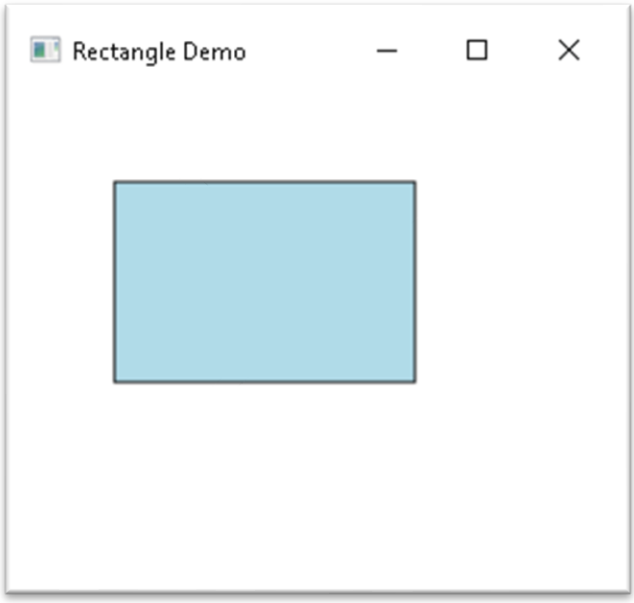
Methods:

getArcHeight() returns the arc height of the rectangle
getArcWidth() returns the arc width of the rectangle
getHeight() returns the height of the rectangle
getWidth() returns the width of the rectangle
getX() Gets the value of the property x.
getY() Gets the value of the property y.
setArcHeight(double v) sets the arc height of the rectangle
setArcWidth(double v) sets the arc width of the rectangle
setHeight(double value) sets the height of the rectangle
setWidth(double value) sets the width of rectangle
setX(double value) set the x coordinate of position of rectangle
setY(double value) set the y coordinate of position of rectangle
For example,

```
import javafx.application.Application;
import javafx.scene.Scene;
```

```java
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.stage.Stage;

public class DrawRectangle extends Application {
    @Override
    public void start(Stage primaryStage) {
        Pane pane = new Pane();

        // Create a rectangle (x, y, width, height)
        Rectangle rect = new Rectangle(50, 50, 150, 100);
        rect.setFill(Color.LIGHTBLUE);
        rect.setStroke(Color.BLACK);
        rect.setStrokeWidth(1);

        pane.getChildren().add(rect);
        Scene scene = new Scene(pane, 300, 250);
        primaryStage.setTitle("Rectangle Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:



Drawing Text

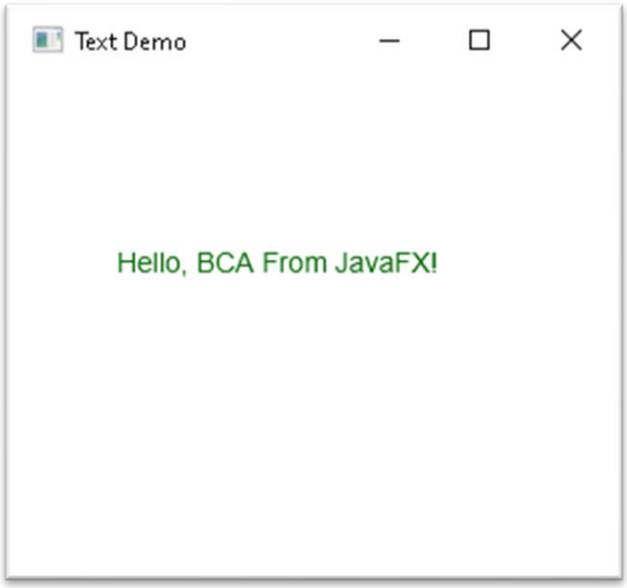
Use the Text class to display text on the screen.

For example,

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.text.Font;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class DrawText extends Application {
   @Override
   public void start(Stage primaryStage) {
      Pane pane = new Pane();
```

```
        Text text = new Text(50, 100, "Hello, BCA From JavaFX!");
        text.setFont(new Font("Arial", 14));
        text.setFill(Color.DARKGREEN);

        pane.getChildren().add(text);
        Scene scene = new Scene(pane, 300, 250);
        primaryStage.setTitle("Text Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:



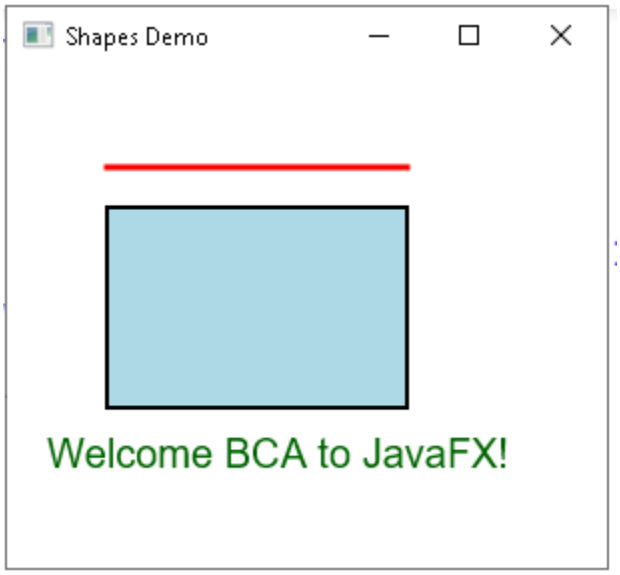
Combining Line, Rectangle, and Text all together

You can add multiple shapes to a single Pane to display them together.

```
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.layout.Pane;
import javafx.scene.paint.Color;
import javafx.scene.shape.Line;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Font;
import javafx.scene.text.Text;
import javafx.stage.Stage;

public class DrawShapes extends Application {
    @Override
    public void start(Stage primaryStage) {
        Pane pane = new Pane();

        // Line
        Line line = new Line(50, 50, 200, 50);
        line.setStrokeWidth(3);
        line.setStroke(Color.RED);

        // Rectangle
        Rectangle rect = new Rectangle(50, 70, 150, 100);
        rect.setFill(Color.LIGHTBLUE);
        rect.setStroke(Color.BLACK);
        rect.setStrokeWidth(2);

        // Text
```

```
        Text text = new Text(20, 200, "Welcome BCA to JavaFX!");
        text.setFont(new Font("Arial", 20));
        text.setFill(Color.DARKGREEN);

        pane.getChildren().addAll(line, rect, text);
        Scene scene = new Scene(pane, 300, 250);
        primaryStage.setTitle("Shapes Demo");
        primaryStage.setScene(scene);
        primaryStage.show();
    }
    public static void main(String[] args) {
        launch(args);
    }
}
```

Output:



Project Work: Develop a Project using JavaFX and Database for CREATE, READ, UPDATE and DELETE operation and send to sunil@nast.edu.np.

<<Best wishes>>